

Task1

Executive Summary

The Conduit application was evaluated across its core user flows (Sign-up, Login, Article Creation, Editing, Favoriting, and Logout). While the application establishes a solid conceptual baseline, several critical functional, usability, and security vulnerabilities currently block it from being production-ready. Fixing these issues will drastically lower user abandonment rates during signup and increase general platform engagement.

Part 1: Bug & UX Assessment Table

Executive Summary

The Conduit application was evaluated across its core user flows (Sign-up, Login, Article Creation, Editing, Favoriting, and Logout). While the application establishes a solid conceptual baseline, several critical functional, usability, and security vulnerabilities currently block it from being production-ready. Fixing these issues will drastically lower user abandonment rates during signup and increase general platform engagement.

Part 2: Root-Cause Analysis (RCA)

Target Issue: BUG-004 — Persistent Session Security Risk (Authentication)

What is Happening

When a user logs into Conduit, the application successfully authenticates the credentials and stores a JSON Web Token (JWT) directly inside the browser's localStorage. However, when closing the browser tab or restarting the device, the user remains permanently logged in. There is no built-in mechanism to check if the session has been idle for too long or to force a re-login after a designated time window.

Why it is Happening

The root cause is a lack of session-handling logic on the frontend coupled with an infinitely long or non-expiring lifespan assigned to the JWT on the backend server. The app is designed to look at `localStorage.getItem('jwt')` upon loading. If that token exists, it assumes the user is authenticated. It does not verify whether the token has passed a safe expiration timestamp before granting access to personal feeds and account settings.

How to Fix It

To fix this, we need to implement a two-pronged solution:

1. Backend Adjustments: Configure the authentication server to issue tokens with a strict Time-To-Live (TTL) expiration window (e.g., 2 hours).
2. Frontend Interceptors: Update our global frontend router to parse the token's expiration date before rendering protected routes. If the current time is greater than the token's expiration

timestamp, the frontend should automatically clear localStorage and smoothly redirect the user back to the login screen with an explanatory message.

Issue ID	Flow / Area	Technical Problem	User Experience Impact	Severity
BUG-001	User Sign-up	Form allows submission with blank spaces (" ") or missing username properties without rejecting the request locally.	Users can inadvertently create unclickable or completely blank profiles, breaking their public profile page.	High
BUG-002	Article Editor	Missing client-side markdown preview or formatting toolbar; text area relies entirely on raw text entry.	Non-technical writers have no way of knowing how their article looks until it is published, causing frustration	Medium
BUG-004	Authentication	The application retains user session states in local storage indefinitely without an automated token expiration check.	If a user signs in on a public or shared computer, the next person using that browser remains logged into their account.	High
BUG-005	Form Validation	Failed network requests (e.g., trying to register an already taken email) return raw API error strings instead of friendly text.	The app displays cryptic messages like "email has already been taken" in raw unstyled text,	Low

			making the app look broken.	
--	--	--	-----------------------------	--

Part 2: Root-Cause Analysis (RCA)

Target Issue: BUG-004 — Persistent Session Security Risk (Authentication)

What is Happening

When a user logs into Conduit, the application successfully authenticates the credentials and stores a JSON Web Token (JWT) directly inside the browser's `localStorage`. However, when closing the browser tab or restarting the device, the user remains permanently logged in. There is no built-in mechanism to check if the session has been idle for too long or to force a re-login after a designated time window.

Why it is Happening

The root cause is a lack of session-handling logic on the frontend coupled with an infinitely long or non-expiring lifespan assigned to the JWT on the backend server. The app is designed to look at `localStorage.getItem('jwt')` upon loading. If that token exists, it assumes the user is authenticated. It does not verify whether the token has passed a safe expiration timestamp before granting access to personal feeds and account settings.

How to Fix It

To fix this, we need to implement a two-pronged solution:

1. Backend Adjustments: Configure the authentication server to issue tokens with a strict Time-To-Live (TTL) expiration window (e.g., 2 hours).
2. Frontend Interceptors: Update our global frontend router to parse the token's expiration date before rendering protected routes. If the current time is greater than the token's expiration timestamp, the frontend should automatically clear `localStorage` and smoothly redirect the user back to the login screen with an explanatory message.